Chapter 3 · SQL: rows, groups and nested questions

A shop records purchases. Asha has two purchases, Ravi one. Joining customers to purchases gives Asha two output rows. The join has produced purchase matches, not one row per customer. This distinction explains many count questions.

Vocabulary. A query block is one SELECT statement with its clauses. An alias names a table's role. A group collects rows with the same grouping values. An aggregate summarizes those rows. A correlated subquery refers to an outer query's row.

R09 Trace a query block

Use logical evaluation order as a reasoning aid: FROM/JOIN and ON, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, then the stated row limit. The engine may use another execution plan.

1. Produce matching rows, preserving SQL duplicates.
2. Keep only rows where WHERE is TRUE.
3. Form groups and compute aggregates.
4. Apply HAVING, then calculate selected expressions.
5. Remove duplicates only if DISTINCT or a set operator requires it.

UNION normally removes duplicates; UNION ALL preserves them. Do not apply relational-algebra projection semantics to ordinary SQL SELECT.

Worked example W09. Customer contains (1,Asha),(2,Ravi). Purchase contains (1,10),(1,20),(2,30). Joining on the ID produces Asha twice and Ravi once. WHERE amount>15 leaves (Asha,20),(Ravi,30). SELECT Name returns two rows. Without that WHERE, SELECT Name returns Asha,Asha,Ravi; SELECT DISTINCT Name returns Asha,Ravi. Trace a query block in stages even if the database executes it differently. Classical algebra and ordinary SQL disagree about duplicate removal here.

Check before moving on. Identify duplicate-removal clauses explicitly. SELECT alone does not remove duplicates.

R10 NULL and aggregate questions

NULL indicates missing/unknown information. A comparison with NULL ordinarily produces UNKNOWN, including NULL = NULL. Use IS NULL to test it. WHERE keeps TRUE, excluding FALSE and UNKNOWN.

Write the three-valued truth table when needed. FALSE AND UNKNOWN is FALSE; TRUE OR UNKNOWN is TRUE. COUNT(*) counts rows. COUNT(column) counts non-null values. COUNT(DISTINCT column) counts distinct non-null values. SUM and AVG ignore NULL inputs. On empty input, COUNT returns zero and ordinary SUM/AVG return NULL. An ungrouped aggregate still produces one row; GROUP BY on empty input produces no groups.

Worked example W10. For Value={10,NULL,10,20}, COUNT(*)=4, COUNT(Value)=3, COUNT(DISTINCT Value)=2, SUM(Value)=40 and AVG(Value)=40/3. The average divides by three observed numbers, not four rows. On no matching rows, an ungrouped aggregate returns one row containing COUNT=0 and SUM=NULL. A grouped aggregate returns no groups. NULL=10 and NULL=NULL are UNKNOWN, not FALSE. WHERE accepts only TRUE. FALSE AND UNKNOWN is FALSE; TRUE AND UNKNOWN is

UNKNOWN; TRUE OR UNKNOWN is TRUE; FALSE OR UNKNOWN is UNKNOWN; NOT UNKNOWN stays UNKNOWN.

Check before moving on. Write the denominator of AVG and distinguish an empty group from an absent group.

R11 GROUP BY and HAVING

Partition the surviving input rows by the full grouping tuple. Write each group's members, then its aggregates. Apply HAVING to the group. Keep the requested output columns. In standard exam SQL, selected nonaggregate columns must be valid grouping columns or otherwise permitted by the question's stated semantics. Do not accept a permissive engine's arbitrary value as an exam rule.

Worked example W11. Sales rows are (A,10),(A,20),(B,7),(B,NULL). WHERE amount>8 leaves only A's two rows. GROUP BY team creates one A group; SUM=30 and COUNT(*)=2. HAVING SUM(amount)>25 keeps that group. Moving the condition into HAVING does not mean filtering individual amounts. If the query instead groups all four rows, B's sum is 7 and its COUNT(amount) is 1, while COUNT(*) is 2. Always write a group's members before its aggregate.

Check before moving on. WHERE tests rows; HAVING tests groups. Write both stages separately.

R12 Scalar and correlated subqueries

First ask whether the inner query depends on the outer row. If not, evaluate it once. If it does, freeze one outer row, substitute its values, evaluate the inner query and test the outer condition. Repeat for each outer row. A scalar subquery expects at most one row; multiple rows are not silently averaged. An aggregate without GROUP BY returns one row, although its value can be NULL.

For ranking by comparisons, count rows strictly above and strictly below each value. Tied rows can affect counts even if they do not satisfy strict inequalities. For balances 5,5,10,15,18, the condition “number above >= number below” retains the first three rows. Do not replace it with a rank shortcut without checking ties.

Worked example W12. For balances 5,5,10,15,18, count values strictly above and below each row. A balance-5 row has 3 above and 0 below; balance 10 has 2 above and 2 below; 15 has 1 above and 3 below; 18 has 0 above and 4 below. The test above≥below retains the two 5 rows and the 10 row: 3 output rows. For an uncorrelated example, SELECT MAX(amount) from one branch gives a single threshold; compare every outer amount with that one value. An aggregate on an empty branch gives NULL, so amount>MAX(empty) keeps no rows.

Check before moving on. Freeze one outer row before evaluating a correlated subquery. Count ties using the exact inequality.

R13 EXISTS IN ANY and ALL

EXISTS tests whether the inner query returns any row. It does not test whether the selected value is non-null. NOT EXISTS tests absence. IN is an equality-membership test and can be UNKNOWN when NULL is involved. NOT IN can therefore fail to retain an apparently unmatched row.

For ANY, test the comparison against each value: any TRUE gives TRUE; otherwise UNKNOWN can remain. For ALL, any FALSE gives FALSE; otherwise UNKNOWN can remain. Over empty input, ANY is FALSE and ALL is TRUE. EXISTS over empty input is FALSE.

Worked example W13. Applicant IDs are 1,2,3. Rejected IDs are 2,NULL. For ID 1, NOT IN tests NOT(1=2 OR 1=NULL): NOT(FALSE OR UNKNOWN) is UNKNOWN. It is filtered out. ID 2 is explicitly rejected, and ID 3 is also UNKNOWN. NOT IN returns no applicants. A correlated NOT EXISTS asks whether a rejected row actually matches the applicant ID. Only 2 has such a row, so it returns 1 and 3. On an empty subquery, EXISTS and ANY are FALSE, while ALL and NOT EXISTS are TRUE. For 5>ALL{2,NULL}, no comparison is FALSE but one is UNKNOWN, so the result is UNKNOWN.

Check before moving on. Check NULL and empty input before replacing NOT IN with NOT EXISTS or ALL with MAX.

R14 Outer joins and self joins

For an outer join, make ON matches first, then add preserved unmatched rows with NULL on the missing side. Apply WHERE afterwards. A WHERE test on the missing side can remove these rows.

For a self join, draw two independent copies of the table and label their aliases. Form pairs satisfying the predicate. The two aliases can refer to the same underlying row unless an inequality excludes that case. Compute per-row matches before grouping. Check whether a predicate intended to match a team instead matches only the same employee.

Worked example W14. Asha has purchases 10 and 20; Ravi has none. LEFT JOIN Purchase ON matching_ID AND amount>15 yields (Asha,20),(Ravi,NULL). Put amount>15 in WHERE instead: the unmatched Ravi row is now rejected because NULL>15 is UNKNOWN. For a self join on team, aliases A and B represent independent employee roles. An employee in a three-person team has three B matches, including itself. Adding A.EmpID=B.EmpID reduces it to one. Grouping by A.EmpID then counts 1 rather than the team size.

Check before moving on. An unmatched outer-join row is added after ON but before WHERE. Self joins need separate role labels.

Previous-year questions, worked

Read the original question and options in the linked official paper. These are original explanations, not replacement question statements. PDF page numbers count from the first page of the downloaded file.

DA-2024-Q31 · Filter a join

Official question, PDF page 19. Published key: **3 to 3**.

The Jaipur references are Raider IDs 2,1,6. Their raid points are 219,250,215, all strictly above 200. Each reference matches one Raider, so three joined rows survive. Hyderabad's qualifying raider is irrelevant because the team filter also has to hold.

DA-2025-Q33 · One scalar threshold

Official question, PDF page 18. Published key: **3 to 3**.

Evaluate the inner branch maximum first: 65,000. The outer values above it are 90,000,80,000,70,000. Exactly three rows qualify. The 65,000 row fails the strict **>** test. The subquery supplies one value, not a relation to join with every account.

DA-2026-Q51 · Team size per employee

Official question, PDF page 34. Published key: **A**.

Group by TeamID: teams 8,7,9 have sizes 3,2,1. A joins these summaries back to each employee, giving sizes 3,3,3,2,2,1. B additionally requires identical EmpID, so each employee only matches itself and gets 1. C groups by the primary key and also gets 1. D's derived table does not expose TeamID, so B.TeamID cannot be referenced. Only A works.

DA-2026-Q60 · Correlated counts and ties

Official question, PDF page 38. Published key: **3 to 3**.

For balances 5000,5000,10000,15000,18000, the strict above/below counts are (3,0),(3,0),(2,2),(1,3),(0,4). Apply the printed count comparison to each outer row. The first three pass, giving 3. The tied 5000 rows remain two separate account rows even though neither is below the other.

CS-2025-set1-Q55 · Membership then minimum

Official question, PDF page 55. Published key: **26 to 26**.

Mark has coach ID 102, which selects team 10. Members of team 10 are player IDs 1 and 3. Their ages are 31 and 26. MIN(age)=26. Player 2's age 24 is smaller but belongs to team 30, so it is not eligible. Follow identifier links before taking the minimum.

CS-2025-set2-Q54 · Existence formulations

Official question, PDF page 42. Published key: **A; C; D**.

A tests for at least one matching course enrolment. C expresses the same condition with COUNT(*)>0. B uses SIZEOF, which is not the SQL construct in the question. The official key also accepts natural join D, which joins on rollno. Important bag-semantics qualification: if repeated (rollno,courseno) enrolment rows

are allowed, D can repeat a student's name while A and C emit it once per Student row. Thus the official intended enrolment interpretation is not an unconditional bag-equivalence theorem. Use DISTINCT or a suitable enrolment key when that distinction matters.

CS-2022-Q56 · No missing required course

Official question, PDF page 38. Published key: **2 to 2**.

Department D01 requires courses C11 and C12. Subtract each student's registered courses from {C11,C12}. S01 and S04 leave an empty difference; S02 and S05 miss C12; S03 misses both. NOT EXISTS retains S01,S04, giving 2. The paper prints NOT EXIST; read the intended NOT EXISTS construct. Course membership, not the student's own department, determines this predicate.

CS-2023-Q61 · Two simultaneous row filters

Official question, PDF page 64. Published key: **2 to 2**.

Apply gender='F' and marks>65 to each displayed row. The two Aliya rows with marks 70 and 80 pass. Swati's 65 fails the strict inequality. COUNT(*) counts the two rows, not one distinct name. Result: 2.

Chapter practice

Original questions. Write the answer and the intermediate reasoning. Each lettered part belongs to the same problem. The separate solutions guide contains hints, steps and a final check.

P09 · NAT plus variation

Customer rows are (1,Asha),(2,Ravi); Purchase rows are (1,10),(1,20),(2,30). (a) Count SELECT Name over the matching join and SELECT DISTINCT Name over it. Also count customers selected by a correlated EXISTS on purchases. (b) Add Purchase (1,40). Recompute all three counts.

P10 · Short answer

Value contains 10,NULL,10,20. (a) Give COUNT(*), COUNT(Value), COUNT(DISTINCT Value), SUM(Value), AVG(Value). (b) Repeat on WHERE Value>100. How many result rows does the ungrouped aggregate have? How many groups does GROUP BY Value have on that empty input?

P11 · Query tracing

Sales(team,amount)={(A,10),(A,20),(B,7),(B,NULL)}. Evaluate SELECT team,SUM(amount) FROM Sales WHERE amount>8 GROUP BY team HAVING SUM(amount)>25. (a) Show rows after WHERE and the final groups. (b) Add (B,30) and repeat. Would COUNT(*) and COUNT(amount) agree before applying WHERE?

P12 · NAT plus variation

Balances are 5,5,10,15,18. Retain an account when the number of balances strictly above it is at least the number strictly below it. (a) List both counts per balance and give the output count. (b) Replace 15 by 10 and repeat. Treat tied accounts as separate rows.

P13 · Query comparison

Applicant IDs are 1,2,3. Rejected IDs are 2,NULL. (a) Compare WHERE ID NOT IN (SELECT ID FROM Rejected) with a matching-ID NOT EXISTS query. (b) Remove the NULL row and compare again. Also give the truth values of 5>ANY(empty) and 5>ALL(empty).

P14 · Query tracing

C={(1,Asha),(2,Ravi)}, P={(1,10),(1,20)}. (a) List C LEFT JOIN P ON C.ID=P.ID AND P.amount>15. Compare moving P.amount>15 to WHERE. (b) A self join has values 1,1,2 on each side. Count ordered pairs for A.value<B.value and for A.value≤B.value.

Before the exam

Before counting, mark bag versus set semantics, NULL, the grouping tuple and each strict inequality. For nested queries, determine whether the inner result is a scalar, a set or an existence test. For an MSQ, test every option independently, including whether it is valid SQL.